Industrial Safety AI

Описание проекта для команды

Проект: система компьютерного зрения для контроля промышленной безопасности по видеопотоку с камер.
Цель: собрать рабочий MVP, который не просто рисует рамки вокруг объектов, а обнаруживает реальные нарушения, подтверждает их во времени, сохраняет доказательства и уведомляет ответственного сотрудника.
Репозиторий: github.com/judogaaa/Industrial-Safety-AI

Коротко: что мы делаем

Мы создаём универсальную систему, которую можно подключить к обычной камере ноутбука, USB-вебкамере, IP/RTSP-камере или видеофайлу. Система в реальном времени анализирует изображение, определяет человека и средства индивидуальной защиты, контролирует опасные зоны и создаёт инцидент, если нарушение действительно продолжается заданное время.
На финальном этапе инференс модели должен выполняться на российских ускорителях LinQ. Поэтому вся архитектура специально строится так, чтобы сейчас использовать локальный ONNX Runtime, а позже заменить только один модуль - LinqProvider. Камеры, правила безопасности, база, Telegram и интерфейс при этом не должны переписываться.

Главная идея продукта

Пользователь открывает веб-панель, подключает камеру, выбирает правила контроля и при необходимости рисует мышкой опасную зону прямо поверх кадра. После этого система непрерывно следит за происходящим.

Пример настроек одной камеры:

• контроль защитной каски - включён;
• контроль сигнального жилета - включён;
• опасная зона - включена;
• подтверждение нарушения - 2 секунды;
• Telegram-уведомления - включены.
Если человек на один кадр «потерял» каску из-за ошибки модели, инцидент не создаётся. Если отсутствие каски подтверждается около 2 секунд, система создаёт один устойчивый инцидент, сохраняет фотографию и отправляет уведомление.

1. Что должно уметь приложение

Универсальные камеры. Встроенная камера MacBook/Windows, USB-вебкамера, RTSP/IP-камера и MP4 должны попадать в один и тот же pipeline.
Детекция людей и СИЗ. Минимум: person, helmet/no_helmet, vest/no_vest. Модель можно взять open source и затем дообучать.
Опасные зоны. Пользователь рисует polygon поверх изображения камеры. Вход человека внутрь зоны становится правилом безопасности.
Tracking. Один человек получает устойчивый ID между кадрами, чтобы система понимала, что Worker #17 в течение нескольких секунд - это один и тот же человек.
Подтверждение во времени. Нарушение должно наблюдаться некоторое время, например 2 секунды, прежде чем станет инцидентом.
Incident Engine. Один длительный проступок = один инцидент, а не сотни алертов по числу кадров. Состояния минимум ACTIVE и RESOLVED.

Evidence. Для каждого инцидента сохраняется кадр с рамками, зоной, временем и типом нарушения.
Telegram. Ответственный сотрудник получает фото, камеру, тип нарушения, время и статус.
Dashboard. Live monitoring, список камер, активные инциденты, история и простая статистика.
Обратная связь. Кнопки «подтвердить» / «ложное срабатывание» позволяют накапливать данные для будущего fine-tuning.
LinQ. На хакатоне локальный inference заменяется LinQ-инференсом без изменения остальной системы.

2. Как работает вся цепочка

Этап

Что происходит

1. Источник

Камера ноутбука, USB, RTSP/IP или видеофайл отдаёт кадр.

2. InferenceProvider

Локально кадр идёт в ONNX Runtime; позже - в LinQ.

3. Normalized detections

Любая модель возвращается к нашему единому формату: class_name, confidence, bbox.

4. Tracking

Объекты связываются между кадрами и получают стабильные ID.

5. Safety Rules

Проверяются каска, жилет, polygon-зона и другие включённые правила.

6. Temporal Validator

Кратковременная ошибка не считается нарушением; событие подтверждается во времени.

7. Incident Engine

Создаётся или обновляется один инцидент ACTIVE; после исчезновения нарушения он становится RESOLVED.

8. Output

Снимок, SQLite, Dashboard и Telegram.

Главный принцип: всё после InferenceProvider не должно зависеть от конкретной модели или железа. Это позволяет безболезненно переключиться на LinQ.

3. Универсальная работа с камерами

У нас не будет отдельной версии под A4Tech и отдельной версии под MacBook. В проекте одна кодовая база.
Обычные камеры пользователя. Для встроенной и USB-камеры наиболее универсальный

путь - браузерный API navigator.mediaDevices.getUserMedia(). Браузер сам показывает доступные камеры устройства. На Mac это может быть встроенная FaceTime-камера, на Windows - встроенная камера и подключённая A4Tech.
Промышленные камеры. Для RTSP/HTTP-потоков и видеофайлов источник обслуживает backend через OpenCV/совместимый адаптер.
Разрешение камеры. Опасные зоны хранятся не в фиксированных пикселях, а в нормализованных координатах от 0 до 1. Поэтому одна логика работает на 720p, 1080p и других разрешениях.
Пример нормализованной зоны:
[(0.21, 0.44), (0.65, 0.42), (0.70, 0.83), (0.18, 0.84)]

4. Что такое опасная зона

Нейросеть не должна сама угадывать, где на предприятии опасная область. Администратор один раз открывает изображение с камеры и мышкой рисует polygon, например вокруг станка, конвейера или зоны работы крана.
После этого правило может звучать так: «если контрольная точка Worker #17 находится внутри зоны более 1 секунды - создать CRITICAL incident». Это обычная геометрия поверх результатов детекции, а не отдельная сложная модель.

5. Как определяем каску и жилет

На первом этапе мы не обучаем большую нейросеть с нуля. Берём совместимый open-source PPE checkpoint/dataset, проверяем лицензию, добиваемся стабильной работы и при необходимости дообучаем на нужных классах.
Критически важно, чтобы стандартный выход модели превращался в наш внутренний формат:
Detection(class_name="no_helmet", confidence=0.94, bbox=(x1, y1, x2, y2))

Тогда Rule Engine ничего не знает о конкретном YOLO, ONNX Runtime или LinQ.

6. Архитектура проекта

Модуль

Ответственность

Camera Layer

Получение кадров из браузерной камеры, USB, RTSP/IP или видео.

InferenceProvider

Единый контракт для LocalOnnxProvider и LinqProvider.

Tracking

Устойчивые ID людей/объектов между кадрами.

Safety Engine

PPE-правила, опасные зоны и другие условия безопасности.

Temporal Validator

Подтверждает нарушение только после заданного времени/серии кадров.

Incident Engine

Дедупликация, ACTIVE/RESOLVED, severity, evidence.

Persistence

SQLite для локальной истории инцидентов и настроек.

Notifications

Telegram и дальнейшие интеграции.

API / WebSocket

FastAPI отдаёт состояние и события интерфейсу.

Frontend

React/Vite: monitoring, cameras, incidents, dashboard.

7. Что берём open source, а что делаем сами

Можно использовать готовое:
• PPE-модель/checkpoint с подходящей лицензией;
• OpenCV для работы с видео;
• ONNX Runtime для локального инференса;
• готовый алгоритм tracking, например ByteTrack или аналог;
• обычные библиотеки FastAPI, React и SQLite.
Наш продуктовый и инженерный слой:
• универсальная Camera Layer;
• сменный InferenceProvider;
• единый формат detections;
• редактор опасных зон;
• Safety Rules;
• подтверждение нарушения во времени;
• Incident Engine с ACTIVE/RESOLVED и дедупликацией;
• evidence/snapshot;
• Telegram workflow;
• панель управления;
• LinQ adapter.
В репозитории есть THIRD_PARTY.md. Любой внешний код, модель или датасет должны быть записаны туда вместе с лицензией.

8. Как подключаем LinQ

Сейчас проект работает через LocalOnnxProvider. Когда организаторы дадут SDK/API/доступ к LinQ, реализуется LinqProvider. Он должен вернуть точно такие же нормализованные Detection-объекты.
После этого переключение должно сводиться к конфигурации типа:
INFERENCE_PROVIDER=linq

Камеры, tracking, polygon-зоны, incident logic, SQLite, Telegram и интерфейс не меняются.
Это главный способ не потерять время на хакатоне.

9. Интерфейс

Первая полноценная версия содержит четыре основных экрана:

1. Dashboard. Камеры онлайн, активные инциденты, количество событий сегодня, базовая

статистика.

2. Monitoring. Live-видео с рамками, Worker ID, статусом каски/жилета,

polygon-зонами,

FPS и latency.

3. Cameras. Добавление камеры, выбор источника, переключатели правил, настройка времени подтверждения, рисование зон.

4. Incidents. Фото, камера, время, тип нарушения, длительность, confidence, статус ACTIVE/RESOLVED.

10. Telegram

При подтверждённом нарушении ответственному сотруднику должно приходить сообщение примерно такого вида:
Нарушение промышленной безопасности
Тип: отсутствует защитная каска
Камера: Цех №1
Зона: Станок №4
Worker: #17
Время: 14:52:41
Длительность: 3.2 сек.
Уверенность: 94%
К сообщению прикладывается фотография нарушения.
Желательные кнопки: «Подтвердить», «Ложное срабатывание», «Открыть инцидент».
Вердикт оператора сохраняется и потом может использоваться для улучшения модели.

11. Что входит в первый MVP

Функция

Статус для MVP

Камера ноутбука / USB

Обязательно

MP4

Обязательно

RTSP/IP

Желательно до финальной версии

Person detection

Обязательно

Helmet / no_helmet

Обязательно

Vest / no_vest

Обязательно

Polygon danger zone

Обязательно

Tracking

Обязательно

Подтверждение около 2 секунд

Обязательно

ACTIVE / RESOLVED + дедупликация

Обязательно

Snapshot

Обязательно

SQLite

Обязательно

Telegram

Обязательно

Dashboard

Обязательно

LinQ inference

Обязательно на хакатоне

Fine-tuning pipeline

Если останется время

Что специально НЕ делаем в первой версии
• распознавание лиц;
• десятки типов СИЗ;
• сложный VLM-анализ каждого кадра;
• прогнозирование аварий на много шагов вперёд;
• Kafka, Kubernetes, микросервисный зоопарк и другую лишнюю инфраструктуру;
• самообучение модели онлайн без контроля человека.
Причина простая: сначала должны железно работать два основных сценария - отсутствие каски и вход в опасную зону.

12. Финальный live-demo

1. Подключаем обычную камеру. Система показывает человека как Worker #17.

2. На человеке каска - статус «Норма».

3. Человек снимает каску. Один случайный кадр ничего не создаёт.

4. Через примерно 2 секунды создаётся один incident «Нет каски».

5. Сохраняется фотография, карточка появляется в Dashboard, Telegram получает уведомление.

6. Человек надевает каску - тот же incident переходит в RESOLVED.

7. Человек входит в заранее нарисованную опасную зону - появляется отдельный CRITICAL incident.

8. На экране показываем реальные FPS и latency LinQ.

Если этот сценарий проходит полностью в реальном времени, ядро продукта

готово.

13. Как работаем вдвоём

Чтобы не мешать друг другу, лучше сразу разделить ответственность, но оба должны уметь запустить весь проект.

Направление	
Основная работа	
Backend / CV	Камеры, inference, PPE, tracking, zones, incidents, LinQ.
Frontend / Product	React-интерфейс, камеры/настройки, monitoring, incidents, dashboard.
Общее	Интеграция, Telegram, тесты на Windows и Mac, live-demo, презентация.

14. GitHub-процесс

У нас один приватный репозиторий: judogaaa/Industrial-Safety-AI. Основная ветка - main. Работать напрямую в main не нужно.

Пример веток:

- feature/camera-layer
- feature/ppe-inference
- feature/safety-rules
- feature/incidents
- feature/dashboard
- feature/telegram
- feature/linq

После того как функция работает локально, открываем Pull Request в main. Так у нас всегда остаётся рабочая версия и можно параллельно работать на Windows и macOS.

15. Что уже есть сейчас

- создан приватный GitHub-репозиторий;
- добавлен FastAPI backend skeleton;
- есть базовый интерфейс InferenceProvider;
- есть заготовки LocalOnnxProvider и LinqProvider;
- есть backend FrameSource для видео/RTSP;
- есть единый формат Detection;
- добавлены базовые тесты и CI для Linux, Windows и macOS;
- зафиксированы архитектура, roadmap и политика third-party лицензий.

Следующий практический этап - сделать универсальную camera layer и проверить один и тот же commit на двух машинах: Windows + A4Tech и MacBook + встроенная камера. После этого подключаем настоящую PPE-модель.

16. Когда считаем проект готовым

MVP считается готовым не тогда, когда открыт красивый сайт, а когда одна и та же версия кода стабильно проходит полный сценарий на разных компьютерах и камерах:
камера -> детекция -> tracking -> правило -> подтверждение -> один incident -> snapshot -> Telegram -> RESOLVED.

Если этот pipeline работает локально и затем тот же pipeline работает через LinQ, задача решена правильно.